In [1]:
```
import pandas as pd
df = pd.read_csv("./boston_housing.csv")
print(df.head())
df.head().to_html("./example2_1.html")
```

```
      crim    zn  indus  chas    nox     rm   age     dis  rad  tax  ptratio  \
0  0.00632  18.0   2.31     0  0.538  6.575  65.2  4.0900    1  296     15.3
1  0.02731   0.0   7.07     0  0.469  6.421  78.9  4.9671    2  242     17.8
2  0.02729   0.0   7.07     0  0.469  7.185  61.1  4.9671    2  242     17.8
3  0.03237   0.0   2.18     0  0.458  6.998  45.8  6.0622    3  222     18.7
4  0.06905   0.0   2.18     0  0.458  7.147  54.2  6.0622    3  222     18.7

        b  lstat  medv
0  396.90   4.98  24.0
1  396.90   9.14  21.6
2  392.83   4.03  34.7
3  394.63   2.94  33.4
4  396.90   5.33  36.2
```

In [2]:
```
print(df.describe())
df.describe().to_html("./example2_2.html")
```

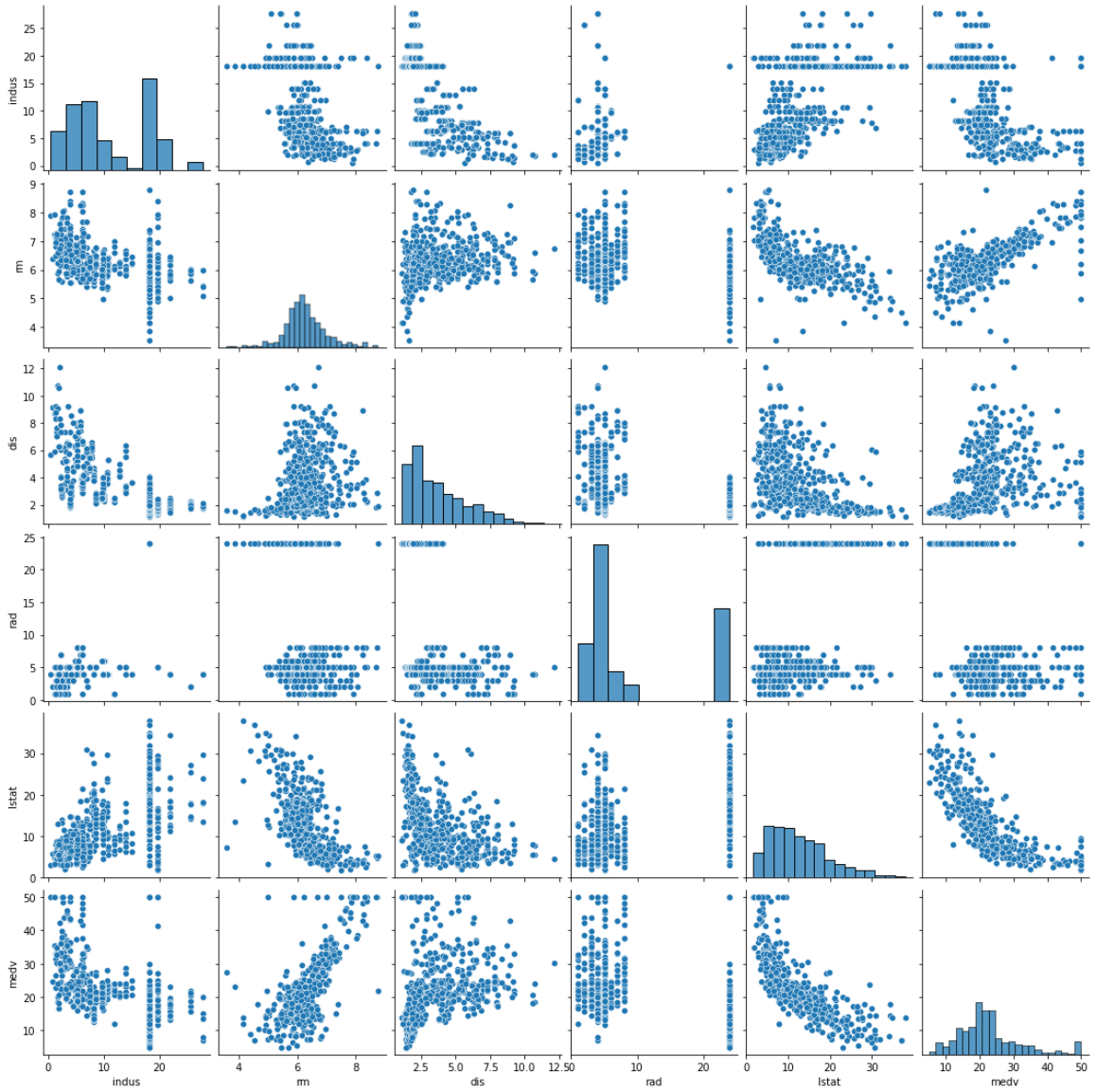
```
             crim          zn       indus        chas         nox          rm  \
count  506.000000  506.000000  506.000000  506.000000  506.000000  506.000000
mean     3.613524   11.363636   11.136779    0.069170    0.554695    6.284634
std      8.601545   23.322453    6.860353    0.253994    0.115878    0.702617
min      0.006320    0.000000    0.460000    0.000000    0.385000    3.561000
25%      0.082045    0.000000    5.190000    0.000000    0.449000    5.885500
50%      0.256510    0.000000    9.690000    0.000000    0.538000    6.208500
75%      3.677083   12.500000   18.100000    0.000000    0.624000    6.623500
max     88.976200  100.000000   27.740000    1.000000    0.871000    8.780000

              age         dis         rad         tax     ptratio           b  \
count  506.000000  506.000000  506.000000  506.000000  506.000000  506.000000
mean    68.574901    3.795043    9.549407  408.237154   18.455534  356.674032
std     28.148861    2.105710    8.707259  168.537116    2.164946   91.294864
min      2.900000    1.129600    1.000000  187.000000   12.600000    0.320000
25%     45.025000    2.100175    4.000000  279.000000   17.400000  375.377500
50%     77.500000    3.207450    5.000000  330.000000   19.050000  391.440000
75%     94.075000    5.188425   24.000000  666.000000   20.200000  396.225000
max    100.000000   12.126500   24.000000  711.000000   22.000000  396.900000

            lstat        medv
count  506.000000  506.000000
mean    12.653063   22.532806
std      7.141062    9.197104
min      1.730000    5.000000
25%      6.950000   17.025000
50%     11.360000   21.200000
75%     16.955000   25.000000
max     37.970000   50.000000
```

In [3]:
```
import seaborn as sns
df2=df.drop(["tax","b","nox","chas","crim","ptratio","age","zn"],axis=1)
sns.pairplot(df2)
```

Out[3]: <seaborn.axisgrid.PairGrid at 0x17bdeab0040>



In [4]:
```
#載入numpy函式庫(套件)
import numpy as np
seed=5811
np.random.seed(seed)
df = pd.read_csv("./boston_housing.csv")
dataset = df.values
np.random.shuffle(dataset)
```

In [5]:
```
print("#2-4-2 分割資料集")
#分割成特徵資料和標籤資料
#取0-13的欄位資料，共13個欄位的值(輸入資料)
X = dataset[:, 0:13]
#取編號為13的欄位資料: 第14個欄位的值(輸出資料；自住房屋的中位數價格)
Y = dataset[:, 13]
```

```
#2-4-2 分割資料集
```

In [6]:
```
#特徵標準化
X -= X.mean(axis=0)
X /= X.std(axis=0)

#分割訓練和測試資料集
#訓練資料前404筆
X_train, Y_train = X[:404], Y[:404]
#測試資料後102筆
X_test, Y_test = X[404:], Y[404:]
```

In [7]:
```
X_train.shape, X_test.shape, Y_train.shape, Y_test.shape
```

Out[7]:
```
((404, 13), (102, 13), (404,), (102,))
```

In [8]:
```
def build_model():
    model = Sequential()
    model.add(Dense(32, input_shape=(X_train.shape[1],), activation="relu"))
    model.add(Dense(1))
    model.compile(loss="mse", optimizer="adam",  metrics=["mae"])
    return model
```

In [9]:
```
def build_deep_model():
    model = Sequential()
    model.add(Dense(32,input_shape=(X_train.shape[1],), activation="relu"))
    model.add(Dense(16, activation="relu"))
    model.add(Dense(1))
    model.compile(loss="mse", optimizer="adam",   metrics=["mae"])
    return model
```

In [10]:
```
k = 4
nb_val_samples = len(X_train) // k
nb_epochs = 80
mse_scores = []
mae_scores = []
print(nb_val_samples)
```

```
101
```

In [11]:
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

for i in range(k):
    print("Processing Fold #" + str(i))
    X_val = X_train[i*nb_val_samples: (i+1)*nb_val_samples]
    Y_val = Y_train[i*nb_val_samples: (i+1)*nb_val_samples]
    X_train_p = np.concatenate(
            [X_train[:i*nb_val_samples],
            X_train[(i+1)*nb_val_samples:]], axis=0)
    Y_train_p = np.concatenate(
            [Y_train[:i*nb_val_samples],
            Y_train[(i+1)*nb_val_samples:]], axis=0)
    model = build_model() #or model = build_deep_model()
    model.fit(X_train_p, Y_train_p,
              epochs=nb_epochs, batch_size=16,
              verbose=0)
    mse, mae = model.evaluate(X_val, Y_val)
    mse_scores.append(mse)
    mae_scores.append(mae)
```

```
Processing Fold #0
4/4 [==============================] - 0s 509us/step - loss: 19.1581 - mae: 3.0583
Processing Fold #1
4/4 [==============================] - 0s 736us/step - loss: 18.1981 - mae: 3.3434
Processing Fold #2
1/4 [======>.....................] - ETA: 0s - loss: 14.2485 - mae: 2.9476WARNIN
G:tensorflow:Callbacks method `on_test_batch_end` is slow compared to the batch tim
e (batch time: 0.0000s vs `on_test_batch_end` time: 0.0010s). Check your callbacks.
4/4 [==============================] - 0s 747us/step - loss: 25.5739 - mae: 3.5773
Processing Fold #3
4/4 [==============================] - 0s 1ms/step - loss: 17.8193 - mae: 3.1308
```

In [12]:
```
print("MSE_val: ", np.mean(mse_scores))
print("MAE_val: ", np.mean(mae_scores))
# 使用測試資料評估模型
mse, mae = model.evaluate(X_test, Y_test)
print("MSE_test: ", mse)
print("MAE_test: ", mae)
```

```
MSE_val:  20.187344074249268
MAE_val:  3.2774388790130615
4/4 [==============================] - 0s 997us/step - loss: 19.0128 - mae: 2.9443
MSE_test:  19.012792587280273
MAE_test:  2.944347858428955
```

In [13]:
```
print("#3-5 儲存模型")
#方法一：分開儲存模型結構與權重
json_str = model.to_json()
with open("example2_model.config", "w") as text_file:
     text_file.write(json_str)
model.save_weights("example2_model.weight")
```

```
#3-5 儲存模型
```

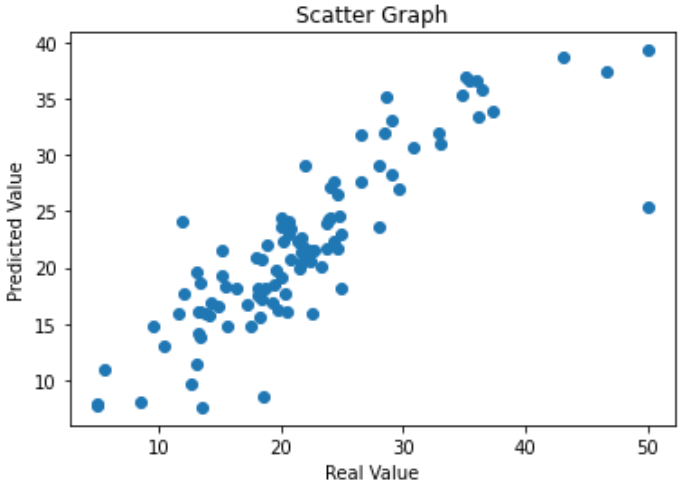
In [14]:
```
model.save("example_2.h5")
```

In [15]:
```
#模型呼叫  方法一：
from tensorflow.keras.models import model_from_json
model = Sequential()
with open("example2_Model.config", "r") as text_file:
    json_str = text_file.read()
model = model_from_json(json_str)
model.load_weights("example2_model.weight", by_name=False)
```

Out[15]:
```
<tensorflow.python.training.tracking.util.CheckpointLoadStatus at 0x17bfc1d3bb0>
```

In [16]:
```
#模型呼叫  方法二：
#from keras.models import load_model
from tensorflow.keras.models import load_model
model = Sequential()
model = load_model("example_2.h5")
```

In [17]:
```
import matplotlib.pyplot as plt   #載入繪圖功能
model.compile(loss="mse", optimizer="adam",
                    metrics=["mae"])    #編譯模型
Y_pred=model.predict(X_test) #預測
plt.title("Scatter Graph")
plt.scatter(Y_test, Y_pred)
plt.xlabel("Real Value")
plt.ylabel("Predicted Value")
plt.show()
```



In [18]:
```
for i in range(len(Y_test)):
    print(Y_test[i], Y_pred[i][0])
```

```
14.1 15.78512
25.0 18.141127
35.1 36.84263
22.0 21.69927
24.4 22.28421
21.7 20.449791
19.8 16.17111
13.5 7.59028
20.7 22.995857
23.7 21.665764
30.8 30.706104
13.4 16.021757
28.1 23.655165
33.1 31.028757
20.5 16.061241
20.3 17.699324
18.4 17.224924
18.9 21.994135
13.3 14.162107
5.0 7.741772
17.2 16.739141
21.7 22.678286
24.8 24.5318
25.0 22.908533
21.9 20.569355
26.6 31.771124
24.4 27.546013
35.4 36.64486
34.9 35.285934
36.5 35.771553
17.5 14.785515
22.4 20.623255
24.1 27.055723
21.4 22.349287
18.4 20.668447
11.9 24.06611
20.1 24.332373
18.3 15.572686
43.1 38.65345
11.7 15.901232
13.4 18.652702
5.0 7.909419
23.8 23.951145
22.0 29.132048
28.0 29.051695
18.7 18.110767
19.3 16.880497
29.1 28.316233
5.6 10.928816
50.0 25.29343
21.7 21.927835
23.9 24.323349
19.6 19.819386
13.1 11.355344
12.1 17.7132
15.6 14.806168
22.6 21.548538
16.4 18.21776
24.7 21.764856
22.5 15.897617
13.4 13.787272
22.7 21.542246
24.1 24.479704
```

```
13.3 16.063957
29.6 26.906937
18.2 17.517815
20.6 24.028492
36.0 36.661423
20.8 20.705477
36.1 33.396584
20.8 23.470499
14.3 16.845152
18.2 18.217472
19.4 18.511219
23.3 20.059172
20.2 22.292665
21.7 21.4023
46.7 37.31704
50.0 39.280663
26.6 27.569292
24.7 26.492517
21.5 19.92784
37.3 33.88095
10.5 12.959536
18.0 20.818937
13.1 19.669256
15.4 18.291643
28.5 31.913801
28.7 35.13277
20.1 23.631075
15.2 19.26669
22.0 21.05351
12.7 9.641678
18.6 8.538079
15.2 21.595222
20.0 19.095827
9.6 14.716392
29.1 33.00182
13.8 15.992048
32.9 32.005035
8.5 8.076195
14.9 16.620106
```